

INFO0501

ALGORITHMIQUE AVANCÉE

COURS 3

GRAPHES ALGORITHMES ÉLÉMENTAIRES



UNIVERSITÉ
DE REIMS
CHAMPAGNE-ARDENNE

Pierre Delisle
Département d'Informatique
Septembre 2022

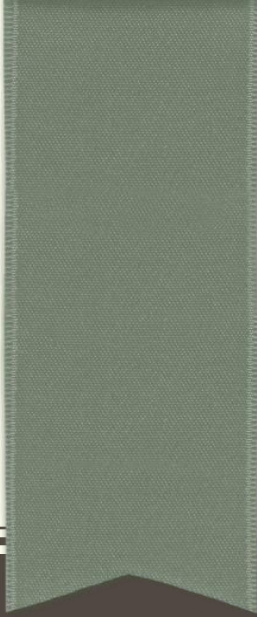
Plan de la séance

- Parcours en largeur
- Parcours en profondeur

- Bibliographie
 - T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, "Algorithmique", 3^e édition, Dunod, 2010

Parcours de graphe ?

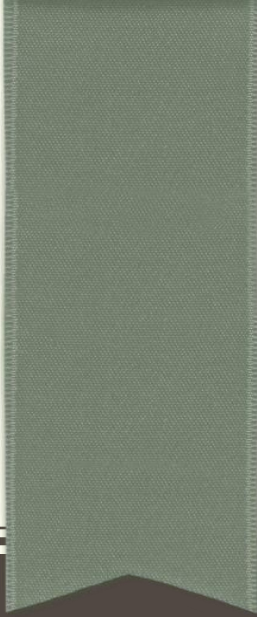
- Sert de base à plusieurs algorithmes
- Permet d'étudier les propriétés du graphe
 - Le graphe est-il connexe ?
 - Le graphe est-il biparti ?
- On pourra aussi faire des traitements sur les sommets et les arcs/arêtes durant le parcours
- 2 types de parcours
 - Largeur
 - Profondeur



PARCOURS DE GRAPHE EN LARGEUR

Parcours en largeur

- Soit un graphe $G = (S, A)$ et un sommet origine s
- Emprunte les arêtes/arcs de G pour découvrir tous les sommets accessibles depuis s
- Calcule la distance (plus petit nombre d'arcs)
 - Entre s et chaque sommet accessible
- Construit un arbre de parcours en largeur
 - De racine s
 - Qui découvre tous les sommets situés à une distance k avant les sommets de distance $k + 1$
- Durant le parcours, chaque sommet devient successivement
 - Non Découvert
 - Découvert
 - Découvert et tous ses sommets adjacents ont été découverts
- Utilise une file pour gérer la découverte des sommets



FLASHBACK

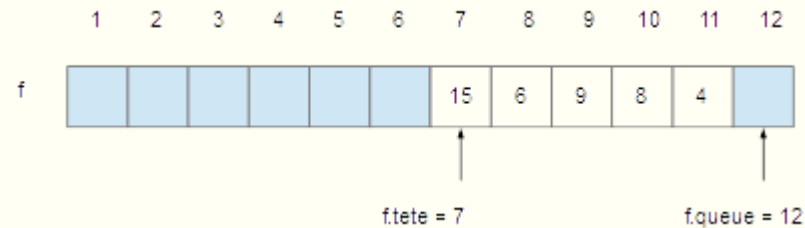
Files

Files

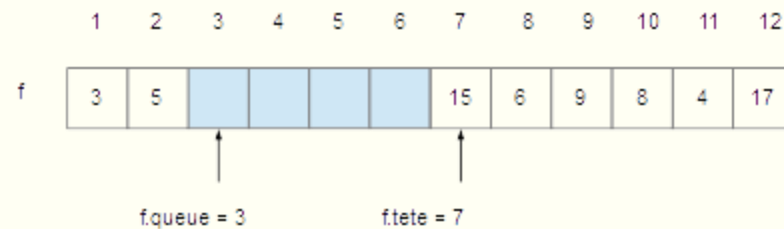
- Premier entré, premier sorti → FIFO
- Insérer → Enfiler
- Supprimer → Défiler
- Implémentation par tableau (circulaire)
 - Au plus $n - 1$ éléments → tableau $f[1...n]$
 - Attribut $f.queue$ qui indexe la queue
 - Enfiler : insertion en $f[f.queue]$
 - Attribut $f.tête$ qui indexe la tête
 - Défiler : suppression en $f[f.tete]$

Files

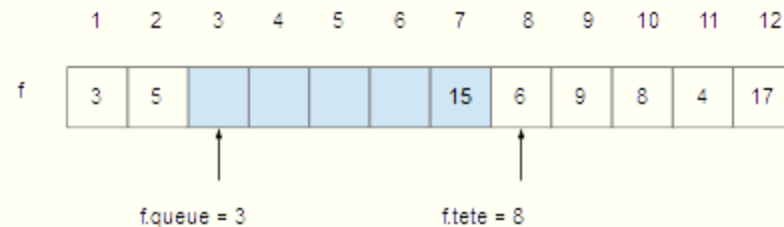
- File contenant 5 éléments



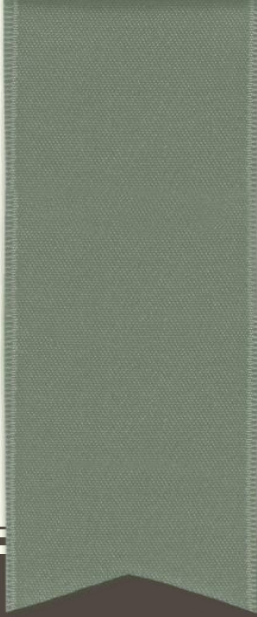
- Après enfilage des valeurs 17, 3 et 5



- Après défilage
 - Retourne la valeur 15
 - L'emplacement 7 n'est plus accessible



- ENFILER (f, x)
 - $O(1)$
- DÉFILER (f)
 - $O(1)$
- RECHERCHER ?

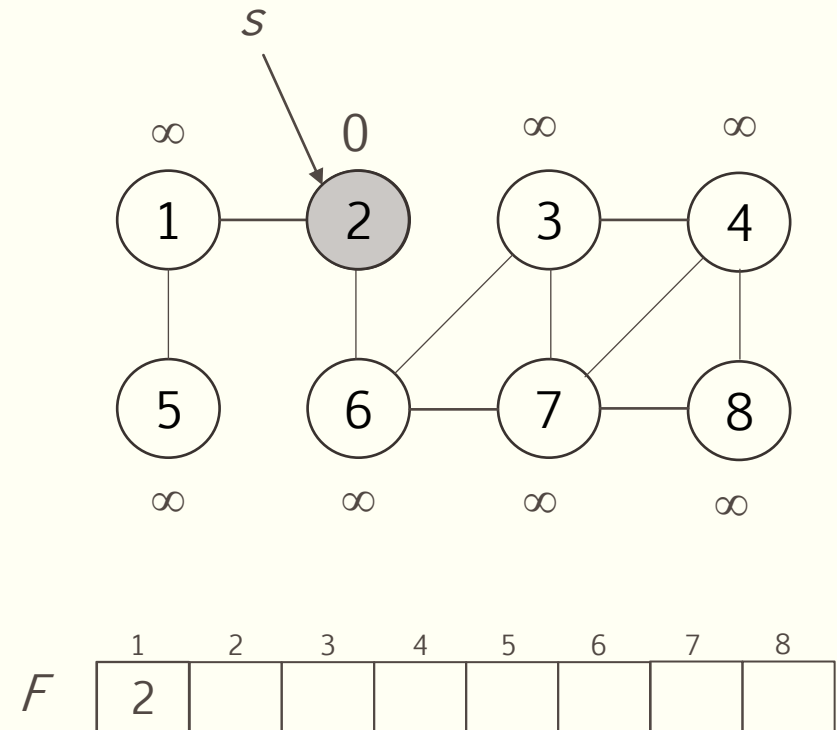


FIN DU FLASHBACK

Files

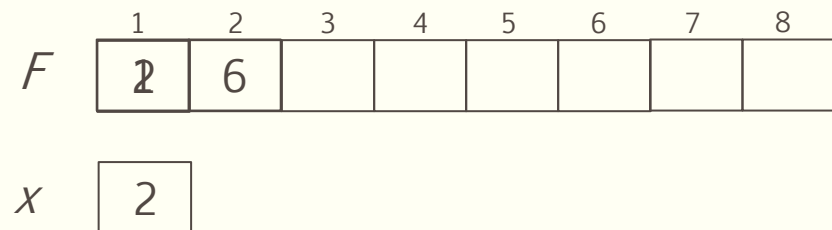
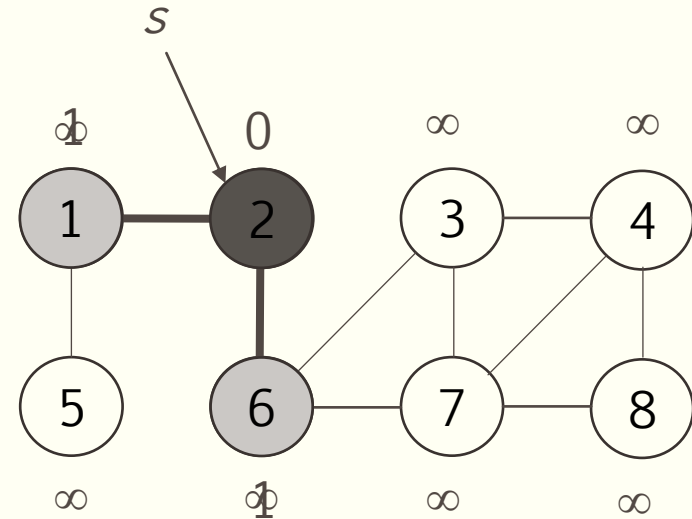
Parcours en largeur – Exemple

- On démarre d'un sommet s spécifique
 - $s = 2$
- On initialise la distance de s à 0 et celle de tous les autres sommets à ∞
- On initialise la couleur de s à GRIS (découvert) et celle de tous les autres sommets à BLANC (non découvert)
- On initialise une file F (vide)
- On enfile le sommet s



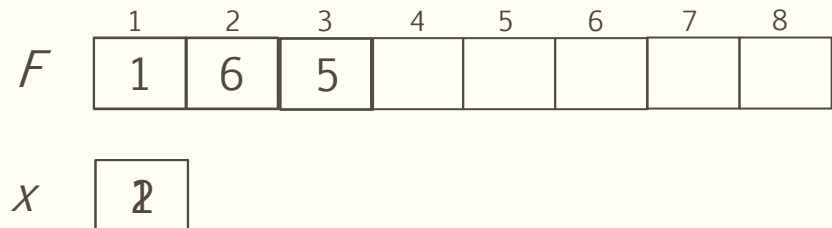
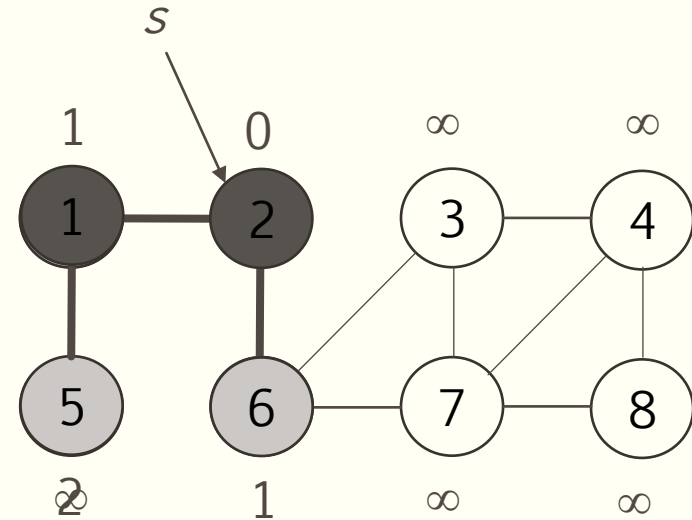
Parcours en largeur – Exemple

- On défile F et on place le sommet défilé dans x
- On découvre (visite) les voisins non visités de x
 - On les colore en GRIS (découvert)
 - On initialise leur distance à la distance de $x + 1$
 - On met à jour leur père qui devient alors x
 - On les enfile dans F
- Tous les voisins de x ont été découverts alors on le colore en NOIR



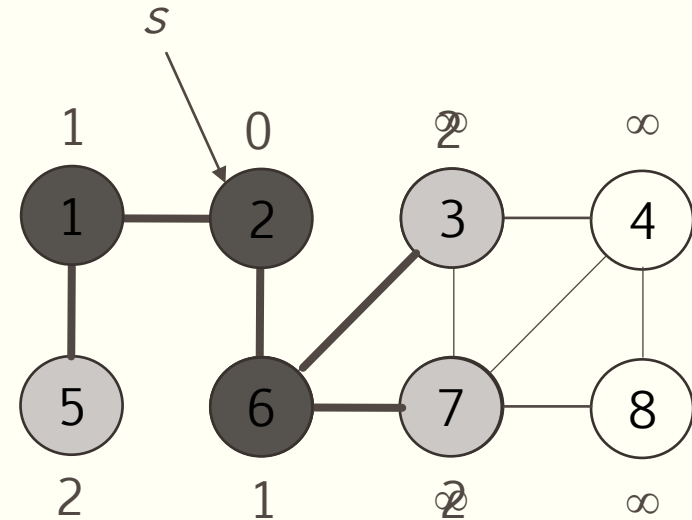
Parcours en largeur – Exemple

- On défile F et on place le sommet défilé dans x
- On découvre (visite) les voisins non visités de x
 - On les colore en GRIS (découvert)
 - On initialise leur distance à la distance de $x + 1$
 - On met à jour leur père qui devient alors x
 - On les enfile dans F
- Tous les sommets de x ont été découverts alors on le colore en NOIR



Parcours en largeur – Exemple

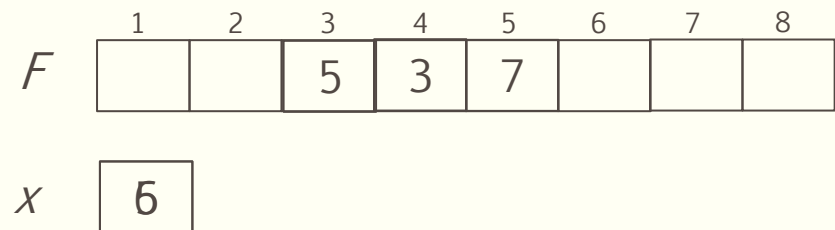
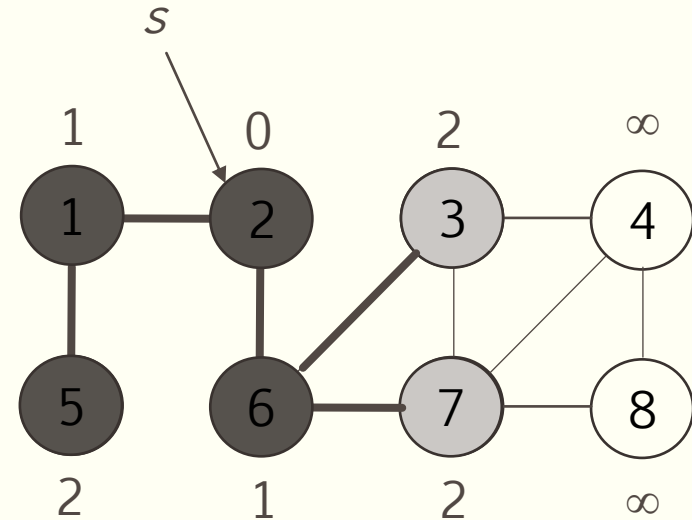
- On défile F et on place le sommet défilé dans x
- On découvre (visite) les voisins non visités de x
 - On les colore en GRIS (découvert)
 - On initialise leur distance à la distance de $x + 1$
 - On met à jour leur père qui devient alors x
 - On les enfile dans F
- Tous les sommets de x ont été découverts alors on le colore en NOIR



	1	2	3	4	5	6	7	8
F		6	5	3	7			
x		6						

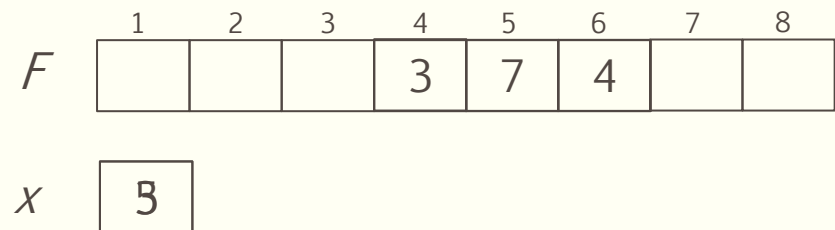
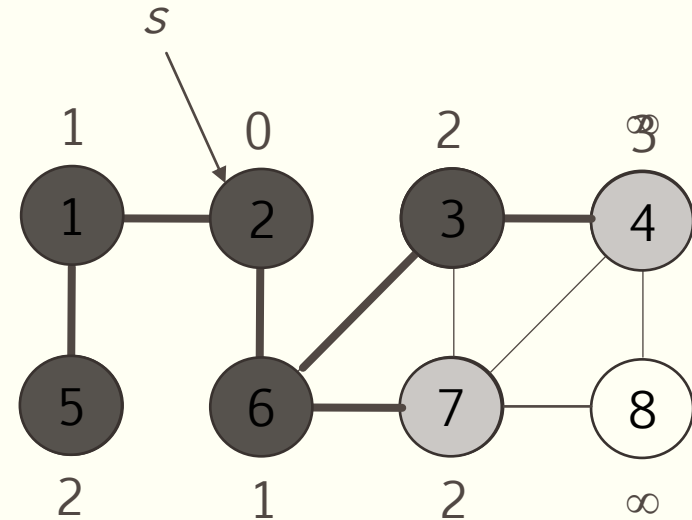
Parcours en largeur – Exemple

- On défile F et on place le sommet défilé dans x
- On découvre (visite) les voisins non visités de x
 - On les colore en GRIS (découvert)
 - On initialise leur distance à la distance de $x + 1$
 - On met à jour leur père qui devient alors x
 - On les enfile dans F
- Tous les sommets de x ont été découverts alors on le colore en NOIR



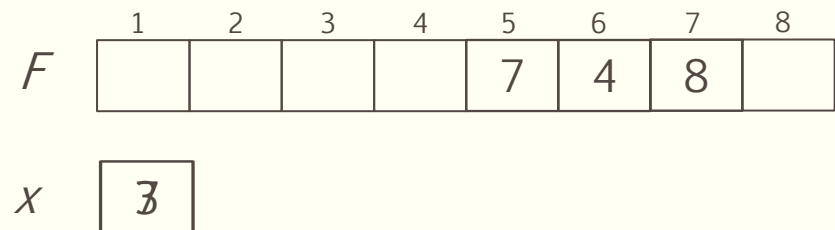
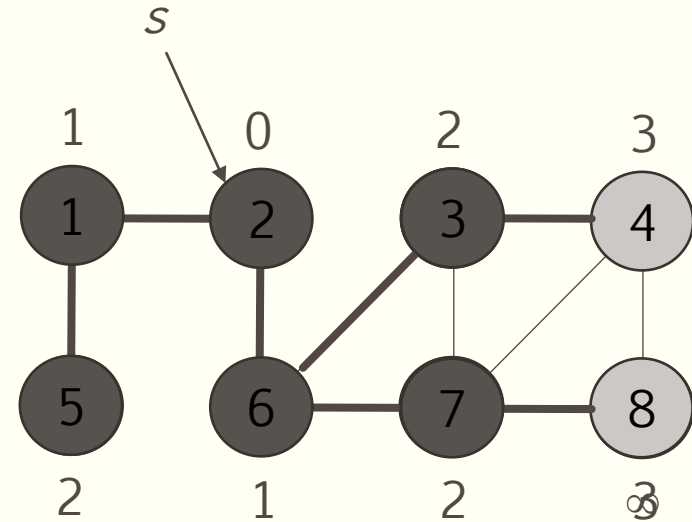
Parcours en largeur – Exemple

- On défile F et on place le sommet défilé dans x
- On découvre (visite) les voisins non visités de x
 - On les colore en GRIS (découvert)
 - On initialise leur distance à la distance de $x + 1$
 - On met à jour leur père qui devient alors x
 - On les enfile dans F
- Tous les sommets de x ont été découverts alors on le colore en NOIR



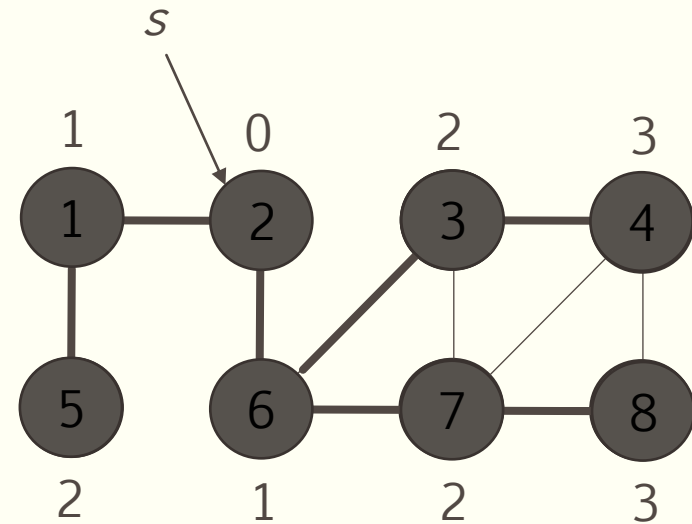
Parcours en largeur – Exemple

- On défile F et on place le sommet défilé dans x
- On découvre (visite) les voisins non visités de x
 - On les colore en GRIS (découvert)
 - On initialise leur distance à la distance de $x + 1$
 - On met à jour leur père qui devient alors x
 - On les enfile dans F
- Tous les sommets de x ont été découverts alors on le colore en NOIR



Parcours en largeur – Exemple

- On défile F et on place le sommet défilé dans x
 - Pas de voisins
 - On colore x en NOIR
- On défile F et on place le sommet défilé dans x
 - Pas de voisins
 - On colore x en NOIR
- F est vide, l'algorithme est terminé
- Les arêtes épaisses représentent l'arbre de parcours en largeur
- Les distances représentent le nombre de sommets qui séparent chaque sommet de s
- Tous les sommets de distance 1 ont été visités, puis ceux de distance 2, puis ceux de distance 3

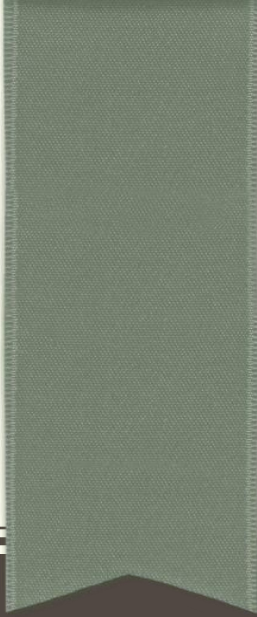


	1	2	3	4	5	6	7	8
F						4	8	

x	8
-----	---

Temps d'exécution du parcours en largeur

- Initialisation de S sommets
 - $\mathcal{O}(S)$
- Chaque sommet est enfilé une fois et défilé une fois
 - $\mathcal{O}(S)$
- Chaque liste d'adjacence est parcourue une fois (quand le sommet est défilé)
 - Somme des longueurs de toutes les listes $\rightarrow \mathcal{O}(A)$
- Total
 - $\mathcal{O}(S + A)$



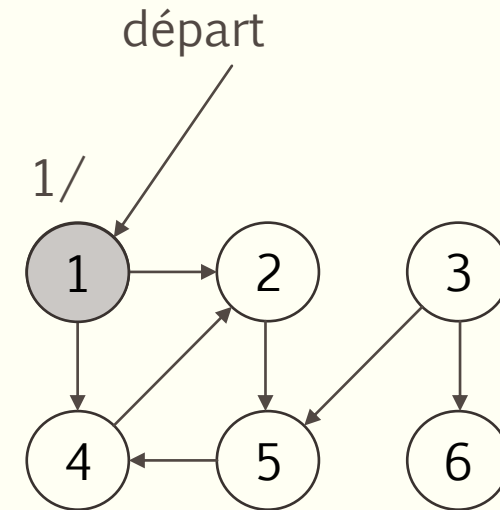
PARCOURS DE GRAPHERS EN PROFONDEUR

Parcours en profondeur

- On descend plus profondément dans le graphe chaque fois que c'est possible
 - On explore les arcs du sommet découvert le plus récemment
 - Si on trouve un sommet non découvert, on l'explore tout de suite même si on n'a pas exploré tous les autres arcs du sommet en cours
 - On revient en arrière plus tard pour explorer les arcs restants
- Construit une forêt de parcours en profondeur
- Le parcours en profondeur date chaque sommet
 - Date de début → découverte du sommet
 - Date de fin → toute la liste d'adjacence du sommet a été examinée

Parcours en profondeur – Exemple

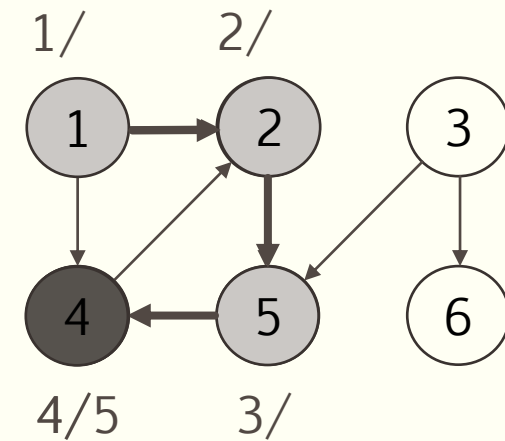
- Pas de sommet de départ prédéfini : on démarre au premier sommet de la liste de sommets
- Une variable globale *date* permet de déterminer les dates de découverte et de fin de traitement
- On initialise la couleur de tous les sommets à BLANC (non découvert)
- On découvre le premier sommet
 - Incrémentation de *date*
 - Date de début : *date*
 - couleur : GRIS (découvert)



date 0

Parcours en profondeur – Exemple

- On visite le premier voisin non visité du sommet courant
 - Incrémentation de *date*
 - Date de début : *date*
 - couleur : GRIS (découvert)
 - *père* : sommet à partir duquel on visite ce voisin
- Sans avoir terminé de visiter les voisins de 1, on visite le premier voisin non visité de 2
- Sans avoir terminé de visiter les voisins de 2, on visite le premier voisin non visité de 5
- Le sommet 4 n'a pas de voisins non visités : fin de traitement
 - Incrémentation de *date*
 - Fin de traitement : *date*
 - Couleur : NOIR

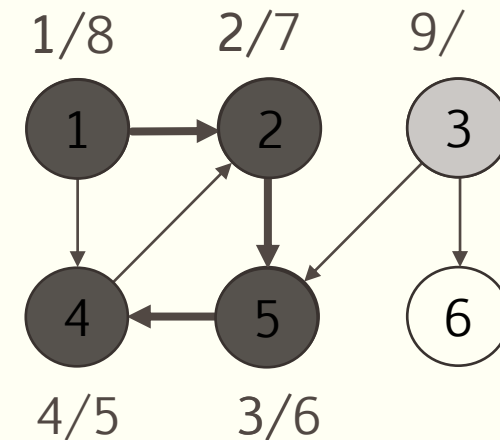


date



Parcours en profondeur – Exemple

- On revient au sommet à partir duquel on a découvert 4 : 5
 - Pas de voisins non visités : fin de traitement
- On revient au sommet à partir duquel on a découvert 5 : 2
 - Pas de voisins non visités : fin de traitement
- On revient au sommet à partir duquel on a découvert 2 : 1
 - Pas de voisins non visités : fin de traitement
- Il n'existe pas de sommet à partir duquel on a découvert 1 : il était au début de cet arbre de parcours
 - On passe donc au prochain sommet non découvert de la liste de sommets : 3

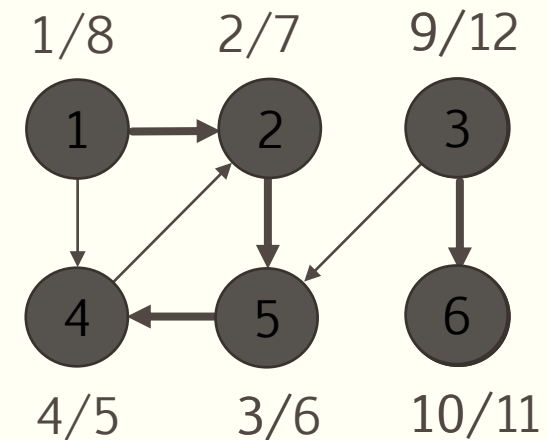


date



Parcours en profondeur – Exemple

- On visite le premier voisin non visité du sommet courant
- Le sommet 6 n'a pas de voisins non visités : fin de traitement
- On revient au sommet à partir duquel on a découvert 6 : 3
 - Pas de voisins non visités : fin de traitement
- Il n'existe pas de sommet à partir duquel on a découvert 3 : il était au début de cet arbre de parcours
 - On passe donc au prochain sommet non découvert de la liste de sommets
 - Fin de la liste de sommets : algorithme terminé

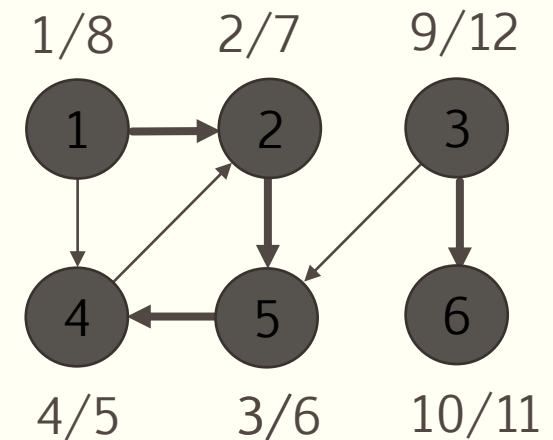


date



Parcours en profondeur – Exemple

- Les dates de début et de fin de traitement indiquent les étapes du parcours en profondeur
- Les arcs épais représentent la forêt de parcours en profondeur
 - On a ici une forêt de 2 arbres
- Ces informations peuvent être utiles pour d'autres algorithmes
 - Détermination de la connexité du graphe
 - Tri topologique
 - ...



date

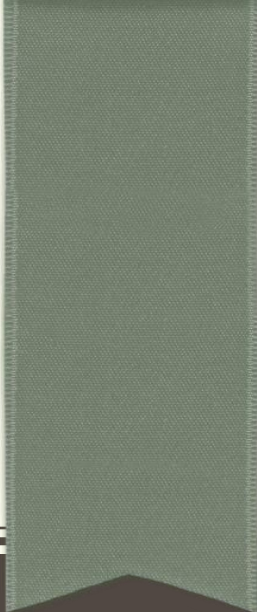
12

Temps d'exécution du parcours en profondeur

- Initialisation de S sommets
 - $O(S)$
- Chaque liste d'adjacence est parcourue une fois
 - Somme des longueurs de toutes les listes $\rightarrow O(A)$
- Total
 - $O(S + A)$

Parcours en profondeur itératif

- Utilise une pile pour gérer la découverte des sommets



FLASHBACK

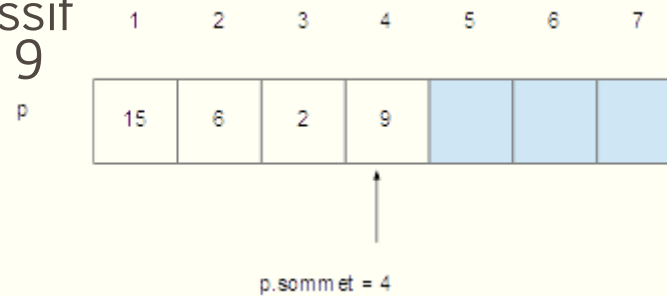
Piles

Piles

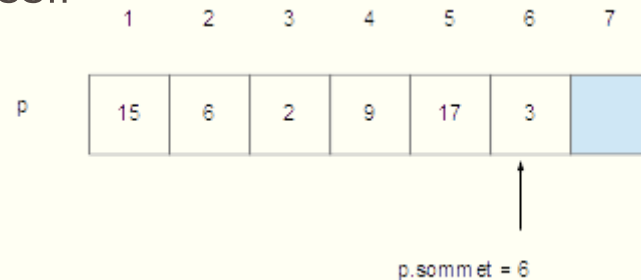
- Dernier entré, premier sorti → LIFO
- Insérer → Empiler
- Supprimer → Dépiler
- Implémentation par tableau
 - Au plus n éléments → tableau p $[1...n]$
 - Possède un attribut $p.sommet$ qui indexe l'élément le plus récemment inséré
 - $p[1]$ → élément situé à la base de la pile
 - $p[p.sommet]$ → élément situé au sommet de la pile

Piles

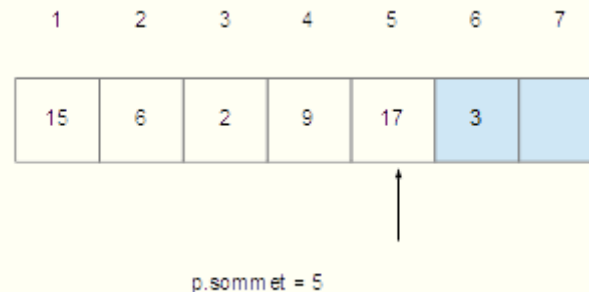
- Après empilage successif des valeurs 15, 6, 2 et 9



- Après empilage successif des valeurs 17 et 3



- Après un dépilage
 - On récupère la valeur 3
 - La case 6 est ensuite non définie, on ne peut pas la réutiliser



- EMPLER (p, x)
 - $\alpha(1)$
- DÉPILER (p)
 - $\alpha(1)$
- PILE-VIDE (p)
 - $\alpha(1)$
- RECHERCHER ?

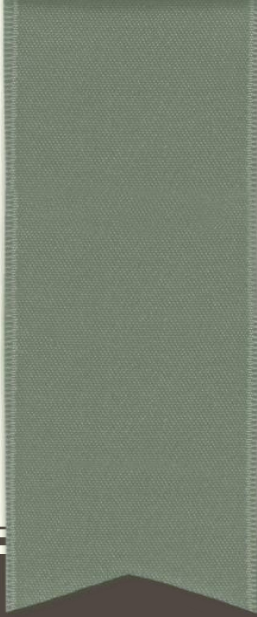
Programmation ?

Java

- Piles
 - Classe Stack (legacy)
- Piles et Files
 - Interface Deque (ArrayDeque, LinkedList)
 - addFirst(..), addLast(..), removeFirst(), removeLast()
- Listes
 - Interface List (ArrayList, LinkedList)
- Tas/Files de priorité
 - Classe PriorityQueue

C++

- Piles
 - Template stack (containers : vector, deque, list)
 - push(..) (push_back), pop() (pop_back), ...
- Files
 - Template queue (containers : deque, list)
 - push(..) (push_back), pop() (pop_front), ...
- Listes
 - Template list (container : doubly linked list)
- Tas/Files de priorité
 - Template priority_queue (cont. : vector, deque)

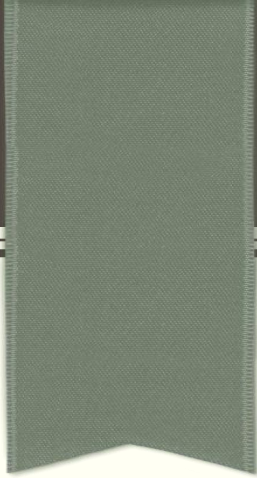


FIN DU FLASHBACK

Piles

Parcours en profondeur itératif – Exemple

- Version simple
 - Pas de dates de début et fin
 - Comportement différent de la version précédente
 - Ordre valide de parcours en profondeur
 - On verra cette version en TD...
- La version itérative avec comportement identique à la version récursive est un peu plus complexe et sera – peut-être – vue en TD



PROCHAIN COURS

ARBRES COUVRANT DE POIDS MINIMUM